WATT-WATCH — Person 3 guide: Appliances, Saving plan, Slides & Demo

> **You are Person 3.** You are responsible for the **appliances and saving plan**, the **slides**, the **demo** and the **backup demo video**.
>
> You **do not need to install or run the code** — Person 1 runs the app. (If you want to run it yourself, see the Appendix at the end.)
>
> **Deadline: 2 days.** Follow the steps in order.

1. What is WATT-WATCH? (read this first)

In Tamil Nadu, electricity is billed in **slabs**: the more units you use in a billing cycle (about 2 months), the higher the price of each extra unit. Families usually find out how much they used only when the bill arrives.

WATT-WATCH is a **web application** (it runs in a browser) that:

```
Scan meter → Track units → Predict bill → Warn before a costlier slab → Reduce usage → Se
e savings
```

Your part is the last two steps: Reduce and Save — plus making the slides and demo.

The team:

Person	Role
Person 1 (team lead)	Runs the app · database · tariff & bills · final report
Person 2	Meter photos (OCR) · prediction · early warning
You (Person 3)	Appliances · saving plan · slides · demo video

2. Your job at a glance

When	Task	Time	Hand in
Day 1 morning	Step A: Get the code & join the team meeting	1 h	—
Day 1 afternoon	Step B: Understand your part	1 h	Your notes
Day 1 afternoon	Step C: Appliance wattage table	45 min	Table → Person 1
Day 1 afternoon	Step D: Saving plan for your home	30 min	Screenshot from Person 1

When	Task	Time	Hand in
Day 1 afternoon	Step E: Make the slides	2 h	Slide deck
Day 1 evening	Step F: Write report section 7.3	1–2 h	Section 7.3
Day 2	Step G: Finish slides, demo video, practise	full day	Slides + video

---

Step A — Get the code and join the team meeting (Day 1 morning)

A1. Download the code (5 minutes)

1. Open **https://github.com/ZumairaSultana/WATTWATCH** in your browser.
2. Near the top left there's a button that says **main** — click it and choose **claude/code-implementation-guide-fqx9ks**. *(The code is on this branch, not on main.)*
3. Click the green **Code** button → **Download ZIP**.
4. Open your **Downloads** folder → right-click the ZIP → **Extract All** → **Extract**. You get a folder whose name starts with **WATTWATCH-**.
5. *(Optional)* Install **VS Code** (code.visualstudio.com) to read the code with colours: **File → Open Folder** → pick that folder.

A2. Team meeting (1 hour)

- Person 1 shows the app. Watch especially the 💡 **Reduce & Save** tab, and notice the order of all 5 tabs — you'll present the demo.
- Person 1 shares the **WATT-WATCH Report** Google Doc with you — open it and find **7.3 Saving plan & interface**.

Step B — Understand your part (1 hour)

Your files:

File	What it does
`wattwatch/appliances.py`	Appliance list, saving plan and savings
`app.py`	The screens: sidebar and the 5 tabs

B1. How much electricity an appliance uses

1 unit = 1 kWh (kilowatt-hour) = using 1000 watts for 1 hour.

```
units per day = watts × quantity × hours per day ÷ 1000
```

Appliance	Watts	Hours/day	Units per day
Air conditioner	1500	6	1500 × 6 ÷ 1000 = **9.0**
Water heater	2000	1	2000 × 1 ÷ 1000 = **2.0**

Appliance	Watts	Hours/day	Units per day
Television	100	5	100 × 5 ÷ 1000 = **0.5**
Ceiling fan × 3	75	10	75 × 3 × 10 ÷ 1000 = **2.25**

The app has a list of common appliances with **typical** wattages (rough values — the real number is on each appliance's label). The user ticks what they use and changes watts, quantity and hours. **Essential** appliances (fridge, Wi-Fi router, mixer) are **never** cut.

B2. The saving plan (`make_saving_plan`)

Goal: save a certain number of units per day with the **fewest hours lost**.

- If Person 2's warning says "cut X units", the daily goal = **X ÷ days left**.
- If there's no warning, the goal is **10%** of daily use.

The method (called "greedy"):

1. Take all **non-essential** appliances, **most powerful first**. *Why: one hour of a 2000 W geyser saves as much as 20 hours of a 100 W TV.*
2. For each one, cut **only the hours still needed** (in half-hour steps)...
3. ...but **at most 50%** of its hours, so the plan stays realistic.
4. Stop when the goal is reached. If every appliance hits its 50% limit first, the app says how much is still missing.

```
for app in sorted(candidates, key=lambda a: a.kw, reverse=True):   # most powerful first
    if remaining <= 1e-9:
        break                                                      # goal reached
    max_cut = _floor_to(app.hours_per_day * max_cut_fraction, step_hours)  # at most 50%
    needed = _ceil_to(remaining / app.kw, step_hours)              # hours still needed
    cut = min(max_cut, needed)
    ...
    remaining -= cut * app.kw
```

Worked example — goal **3.4 units/day**, appliances: water heater 2000 W × 1 h, AC 1500 W × 6 h:

Order	Appliance	Most it can be cut (50%)	Hours cut	Units saved/day	Still needed
1	Water heater (2.0 kW)	0.5 h	0.5 h	2.0 × 0.5 = **1.0**	3.4 − 1.0 = 2.4
2	Air conditioner (1.5 kW)	3 h	2 h	1.5 × 2 = **3.0**	done ✅

Total saved ≈ **4.0 units/day** (a little more than the goal because cuts are in half-hours).

B3. The savings

If the plan is followed for the rest of the cycle (say **32 days left**):

```
units saved  = units saved per day × days left       = 4.0 × 32 = 128 units
money saved  = bill(predicted units) − bill(predicted units − units saved)
CO₂ avoided  = units saved × 0.71 kg                  = 128 × 0.71 ≈ 91 kg
```

Because of the slabs, every unit saved is taken off the **most expensive** slab you were heading into — that's why avoiding a slab jump saves the most money. (0.71 kg CO_2 per unit is an **approximate** figure for India's grid.)

B4. The 5 tabs (`app.py`)

Tab	What it shows
🏠 Households	All households and their latest meter reading
📷 Scan & Track	Upload a meter photo or type a reading; table of all readings
📈 Predict & Warn	Predicted units and bill, chart, early warning
💡 Reduce & Save	Appliance table → saving plan → savings
🧾 Bills	Generate a bill; mark Paid / Unpaid / Pending

The **sidebar** has: household selection, add a household, billing cycle length, tariff rates and other settings.

Step C — Appliance wattage table (45 minutes)

1. At home, find the **sticker or metal plate** on 5–6 appliances (AC, geyser, fridge, fan, TV, water motor, iron box, washing machine). It's usually on the back, side or bottom.
2. Look for a number with **W** (watts), e.g. `1500W`. For an AC it may say *Rated input power*.
3. If it shows only **A** (amps): **watts ≈ 230 × amps** (rough estimate).
4. Ask your family **how many hours a day** each one runs.
5. Fill in and send to Person 1:

Appliance	Watts on label	App's typical watts	Quantity	Hours used per day
Air conditioner		1500		
Water heater		2000		
Refrigerator		100		
Ceiling fan		75		
Television		100		
Motor pump		750		

⚠️ Don't open or unplug anything to read a label. If you can't reach it safely, skip it.

Step D — Saving plan for your home (30 minutes)

1. Send Person 1: your **meter number**, the **reading and date on your last bill**, and **today's reading**.
2. Person 1 adds your home, opens 💡 **Reduce & Save**, ticks your appliances and enters your watts and hours (from Step C).

3. Person 1 sends you a screenshot. Write down: daily goal · which appliances to cut and by how many hours · units saved · money saved · CO_2 avoided.

Step E — Make the slides (2 hours)

Use **Google Slides** (slides.google.com → **Blank presentation**) or PowerPoint. Keep it simple: **one idea per slide, big text, a picture or table on each.**

#	Slide	What to put on it	Presents
1	Title	WATT-WATCH · "Smart electricity bill warning and saving assistant" · team names · guide name · college	Person 1
2	Problem	Slab billing; usage known only when the bill arrives	Person 1
3	Solution & flow	Scan → Track → Predict → Warn → Reduce → Save (as arrows)	Person 1
4	Technology	Python · Streamlit · SQLite · Tesseract OCR	Person 1
5	Database	ER diagram (from Person 1)	Person 1
6	Bill calculation	450-unit slab example + bill check table (from Person 1)	Person 1
7	Scanning (OCR)	Image steps + OCR accuracy (from Person 2)	Person 2
8	Prediction & warning	Worked example + Predict & Warn screenshot (from Person 2)	Person 2
9	Saving plan & savings	B1 formula, B2 example, your home's savings	**You**
10	Live demo	Just the word "Demo" — switch to the app	**You**
11	Results, limitations, future work	3 short lists	**You**
12	Conclusion / Thank you	2–3 lines + "Questions?"	**You**

Screenshots: ask Person 1 for a screenshot of every tab (or take them from the demo video). To take one yourself: **Windows + Shift + S** → drag over the area → paste into the slide with **Ctrl + V**.

Future work ideas for slide 11: an installable mobile app; user logins; SMS/WhatsApp reminders to read the meter; better OCR trained on meter digits; a smarter warning target.

Step F — Write report section 7.3 (evening, 1–2 hours)

Open the Google Doc → **7.3 Saving plan & interface**. Write in your own words:

7.3.1 Appliance energy — 1 unit = 1 kWh; the formula; the table from B1; typical vs label wattages (your Step C table).

7.3.2 Saving plan — the goal (from the warning or 10%); the greedy method and why most powerful first; the 50% limit; essential appliances; the worked example table (B2).

7.3.3 Savings — units, money and CO_2 formulas (B3); your home's results (Step D).

7.3.4 User interface — Streamlit; the sidebar; a short description and screenshot of each of the 5 tabs (B4).

Step G — Day 2

Time	You
9:00 – 11:00	Finish the slides; write report section **10 Future work**.
11:00 – 13:00	With the team: read the whole report, fix mistakes.
14:00 – 15:00	Record the **backup demo video** on Person 1's laptop (below).
15:00 – 17:00	Practise the full presentation **3 times** — you lead the demo.
17:00 – 18:00	Practise the questions (Section 4 below).

How to record the backup demo video

1. On Person 1's laptop, start the app and open it in the browser (full screen: **F11**).
2. Press **Windows + G** → in the *Capture* box click the **record** button (●), or press **Windows + Alt + R** to start/stop.
3. Follow the demo script below, talking as you go.
4. Stop recording. The video is saved in **Videos → Captures**.
5. Copy it to a **pen drive** and Google Drive.

Demo script (you talk, Person 1 clicks) — about 3 minutes

1. 🏠 **Households** — *"These are the households stored in our SQLite database."*
2. Sidebar → select **your home** → 📷 **Scan & Track** → upload a meter photo → **Read number from photo** → check the number → **Add reading** — *"We scan the meter; the user confirms the number."*
3. 📈 **Predict & Warn** — *"We're predicted to use X units this cycle — Rs Y. The app warns us Z days before we cross the next slab."*
4. 💡 **Reduce & Save** — tick appliances — *"If we cut the AC by 2 hours and the geyser by half an hour, we save about N units and Rs M this cycle."*
5. 🧾 **Bills** — **Generate bill** → set **Paid** — *"At the end of the cycle the bill is stored with its payment status."*

If the live demo fails: stay calm, say *"Let me show our recorded demo"*, and play the video.

3. Your part of the presentation (slides 9–12)

Slide	What to say (in your own words)
9. Saving plan	"Each appliance's daily units are watts times hours divided by 1000. To reach the saving goal we cut the most powerful appliances first — one hour of a geyser equals 20 hours of a TV — but never more than half their time, and never the fridge. For my home, the app suggested … saving N units and Rs M."
10. Demo	Run the demo script above.

Slide	What to say (in your own words)
11. Results & limitations	"Bills matched within ...; OCR read X% correctly; predictions for our homes were ... Limitations: OCR needs clear photos, prediction assumes steady use, the tariff must be kept updated."
12. Conclusion	"WATT-WATCH lets families see their bill coming before it arrives and shows exactly how to save. Thank you — any questions?"

4. Questions you may be asked

Question	Answer
Is it an app or a website?	A web application — runs in any browser on a computer or phone, built with Python and Streamlit, data in SQLite.
Why not a mobile app?	A web app works on every device with a browser and was faster to build; an installable mobile app is future work.
How do you calculate an appliance's units?	Watts × quantity × hours ÷ 1000 = kWh = units.
Why cut the most powerful appliances first?	Each hour cut there saves the most units, so the family loses the fewest hours.
Why only 50%?	To keep the plan realistic — people won't stop using an appliance completely.
Why are wattages approximate?	The real value is on each appliance's label; users can change them in the app.
How is CO_2 calculated?	Units saved × about 0.71 kg CO_2 per unit (approximate Indian grid figure).
What is Streamlit?	A Python library that turns Python scripts into web apps without writing HTML.

5. If something goes wrong

Problem	Fix
Can't find the branch on GitHub	Click the button that says **main** near the top left → type `claude` → pick the branch
Windows + G does nothing	Settings → Gaming → Xbox Game Bar → turn it **On**; or use any screen recorder
Recording is only black	Record the browser window (click it first), not the desktop
Live demo fails during presentation	Play the backup video
Missing screenshots/results	Ask Person 1 / Person 2 early on Day 2 morning

Appendix — Run the app on your own laptop (optional, ~45 minutes)

Only if you want to try it yourself (useful for practising the demo, not required).

1. Install **Python** from python.org → ⚠️ tick **"Add python.exe to PATH"** → **Install Now**.
2. Open the extracted **WATTWATCH-...** folder in File Explorer → click the address bar → type **cmd** → **Enter**. *(This opens Command Prompt already inside the folder.)*
3. Type one line at a time:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m pytest
streamlit run app.py
```

4. ✅ Tests show `32 passed`; the browser opens the app at **http://localhost:8501**.
5. To stop: click the Command Prompt window → **Ctrl + C**.